Type Checking Classes and Interfaces

The preceding two sections are all leading up to the very important question of what exactly TypeScript checks when it checks a type.

For example, is the following legal?

```
class Hat {
  constructor(private size: number) {}
}

class Shirt {
  constructor(private size: number) {}
}

let x: Hat;
x = new Shirt()
```

In this code we have two classes, `Hat` and `Shirt`, that are not related to each other but which have the same set of properties, namely an attribute called `size`, which is a number. Given that setup, can you declare a value of type `Hat` and then assign it a value of type `Shirt`?

In many typed languages, this sequence would be an error because the `Hat` and `Shirt` classes have no common ancestor. In TypeScript, though, this is legal.

In TypeScript, types are compared only on their list of properties (the TypeScript docs call this “structural typing”). The basic rule is:

Given an assignment, such as `left = right`, for every property on the left (receiving) side of the assignment, there must be a matching compatible property on the right side of the assignment.

In our earlier example, `Hat` has one property, a number named `size`. `Shirt` has a matching number property named `size`; therefore, a variable of type `Hat` can be assigned a value of type `Shirt`.